TradingLab 2.0 — Guía de la estructura del repositorio

Versión: 1.3

Estado: Documento de apoyo para validación

Fecha: 8 de octubre de 2026

Documento relacionado: `tradinglab-architecture-v2.md`

1. Propósito

Este documento explica qué representa cada elemento de la estructura propuesta del repositorio, qué contenido admite, qué contenido debe evitarse y por qué existe. Su objetivo es que cualquier socio, desarrollador o agente pueda decidir dónde colocar un archivo sin depender de conocimiento informal.

La estructura es un mapa de responsabilidades, no una obligación de crear todas las carpetas desde el primer día. Los directorios se crearán cuando exista contenido real que los justifique.

2. Estructura de referencia

```
trading-lab/
├── README.md
├── AGENTS.md
├── .gitignore
├── .env.example
│
├── .agents/
│   └── skills/
│
├── docs/
│   ├── architecture/
│   ├── adr/
│   ├── knowledge/
│   └── sessions/
│
├── projects/
│   └── <project-slug>/
│       ├── README.md
│       ├── AGENTS.md
│       ├── docs/
│       ├── configs/
│       │   ├── strategy/
│       │   ├── build/
│       │   └── incubation/
│       ├── records/                # registro manual ligero durante el piloto
│       │   ├── strategies/
│       │   └── runs/
│       └── platforms/
│           ├── mt5/
│           │   ├── src/
│           │   ├── include/
│           │   ├── presets/
│           │   └── tests/
│           └── sqx/
│               ├── export-profiles/
│               ├── templates/
│               └── adapters/
│
├── apps/
│   ├── api/
│   ├── cli/
│   ├── worker/
│   └── mt5-runner/
│
├── packages/
│   ├── domain/
│   ├── ingestion/
│   ├── bot-pipeline/
│   ├── exchange-contracts/
│   └── reporting/
│
├── db/
│   ├── migrations/
│   ├── seeds/
```

```
│   └── README.md
│
├── infra/
└── tools/
```

3. Elementos de la raíz

`README.md`

Es la puerta de entrada al laboratorio. Debe permitir que una persona nueva entienda qué es TradingLab 2.0, cómo está organizado y cómo comenzar.

Debe contener:

- propósito y alcance del laboratorio;
- mapa resumido del repositorio;
- requisitos de instalación;
- comandos principales;
- enlaces a arquitectura, ADR y proyectos activos;
- advertencias de seguridad relevantes.

No debe convertirse en un manual exhaustivo ni duplicar documentos especializados. Su función es orientar y enlazar.

`AGENTS.md`

Contiene las reglas globales que deben seguir los agentes de programación al trabajar en el repositorio. Es una política operativa, no documentación comercial.

Debe incluir:

- fuentes de verdad;
- mapa breve del repositorio;
- comandos de build, lint y tests;
- reglas sobre secretos y acciones destructivas;
- criterios para considerar un trabajo terminado;
- skills obligatorias según el tipo de tarea.

Debe ser corto y estable. Los procedimientos extensos pertenecen a skills o documentación. Un `AGENTS.md` dentro de un proyecto solo debe declarar diferencias respecto del archivo raíz.

`.gitignore`

Define qué archivos locales o generados no deben entrar en Git.

Incluirá, entre otros:

- `.env` y variantes con secretos;
- `_storage/` y datasets locales;
- cachés, temporales y logs;
- entornos virtuales y dependencias instaladas;

- builds y ejecutables generados;
- archivos específicos del sistema operativo o IDE;
- reportes masivos generados por MT5/SQX.

No debe utilizarse para ocultar archivos que deberían versionarse pero resultan incómodos. Si un archivo es necesario para reproducir el sistema, debe existir una versión segura y versionable.

`.env.example`

Documenta las variables de entorno que necesita el sistema sin contener valores reales.

Ejemplo:

```
TRADINGLAB_DATABASE_URL=
TRADINGLAB_STORAGE_ROOT=
TRADINGLAB_LOG_LEVEL=INFO
```

Sirve como contrato de configuración y lista de comprobación para nuevas instalaciones. Nunca debe contener contraseñas, tokens, rutas personales o credenciales funcionales.

4. `.agents/` — Procedimientos para agentes

`.agents/skills/`

Contiene workflows reutilizables para agentes como Codex. Una skill describe cuándo y cómo ejecutar una tarea repetitiva del laboratorio.

Ejemplos futuros:

```
.agents/skills/
├── import-mt5-report/
├── import-sqx-results/
├── register-experiment/
├── run-database-migration/
└── close-session/
```

Una skill puede incluir:

```
skill-name/
├── SKILL.md       # activación, instrucciones y resultado esperado
├── scripts/       # operaciones deterministas
├── references/    # formatos y conocimiento auxiliar
└── assets/        # plantillas u otros recursos
```

Las skills no sustituyen la API ni las reglas de seguridad. Enseñan al agente a utilizar capacidades existentes. Solo se crearán después de haber ejecutado manualmente el procedimiento y entender sus excepciones.

5. `docs/` — Conocimiento humano versionado

`docs/` contiene información que debe poder leerse, revisarse y evolucionar mediante Git. No almacena datos operativos ni resultados masivos.

`docs/architecture/`

Describe cómo está construido TradingLab y cómo se relacionan sus componentes.

Contenido típico:

- arquitectura vigente;
- guías explicativas;
- diagramas;
- modelos de dominio;
- contratos técnicos de alto nivel;
- versiones históricas relevantes.

La arquitectura describe el estado o dirección general. Las decisiones individuales y sus alternativas deben registrarse como ADR.

`docs/adr/`

Contiene Architecture Decision Records: registros breves e inmutables de decisiones importantes.

Ejemplos:

- `0001-use-single-repository.md`
- `0002-select-catalog-database.md`
- `0003-artifact-storage-policy.md`

Cada ADR incluye contexto, alternativas, decisión, consecuencias y estado. Cuando una decisión cambia, no se borra la anterior: un nuevo ADR la reemplaza y ambas quedan enlazadas.

`docs/knowledge/`

Reúne conocimiento general aplicable a más de un proyecto.

Ejemplos:

- metodología de validación;
- definiciones de métricas;
- criterios de robustez;
- funcionamiento de MT5 y SQX;
- estándares de calidad de datos;
- glosario de trading.

Si el conocimiento solo aplica a un proyecto, debe vivir en `projects/<slug>/docs/`.

`docs/sessions/`

Es la bitácora cronológica del trabajo realizado.

Cada entrada debe indicar:

- fecha y objetivo;
- material revisado;
- acciones realizadas;
- evidencias y artefactos creados;
- decisiones tomadas;
- preguntas abiertas;
- próximo paso.

La bitácora conserva contexto, pero no sustituye la base de datos, un ADR ni la configuración. Debe enlazar las fuentes autoritativas en lugar de copiar toda su información.

6. `projects/` — Iniciativas de investigación

Cada subdirectorio representa un proyecto con objetivo, alcance y criterios de éxito propios. Un proyecto no equivale necesariamente a una única estrategia: puede agrupar varias estrategias relacionadas.

El nombre `<project-slug>` será estable, descriptivo, en minúsculas y con guiones. Por ejemplo, `scalping-9-robots`.

`projects/<project-slug>/README.md`

Es la ficha principal del proyecto. Incluye:

- objetivo e hipótesis general;
- mercados y timeframes;
- alcance y exclusiones;
- estrategias relacionadas;
- metodología de validación;
- criterios de éxito o abandono;
- estado y responsables;
- enlaces a experimentos y documentación.

`projects/<project-slug>/AGENTS.md`

Es opcional. Solo debe existir cuando el proyecto tenga reglas diferentes de las globales, por ejemplo comandos especiales, formatos propios o restricciones de seguridad adicionales.

No debe copiar el `AGENTS.md` raíz. Las reglas heredadas siguen siendo aplicables.

`projects/<project-slug>/docs/`

Contiene conocimiento exclusivo del proyecto:

- tesis de investigación;
- especificaciones de estrategias;
- interpretación de resultados;
- limitaciones conocidas;

- notas metodológicas específicas.

Una conclusión que luego sea aplicable a todo TradingLab puede promoverse a `docs/knowledge/`, conservando la referencia a su origen.

`projects/<project-slug>/configs/`

Contiene configuraciones pequeñas, legibles y sin secretos que definen experimentos o estrategias.

Ejemplos:

- parámetros de búsqueda;
- ventanas IS/OOS;
- criterios de aceptación;
- configuración normalizada de costes;
- plantillas de ejecución.

Una configuración utilizada debe quedar vinculada a una `StrategyVersion` o un `Run` mediante su hash. Las exportaciones binarias pesadas no pertenecen aquí.

Subdirectorios previstos:

- `strategy/`: parámetros que definen la lógica o versión de estrategia.
- `build/`: recetas de preparación, compilación y gates aplicables al bot.
- `incubation/`: configuración declarativa del deployment y periodo de incubación.

`projects/<project-slug>/records/` — Registro operativo ligero del piloto

Mientras el catálogo todavía no esté implementado, este directorio permite que una persona ejecute el proceso manual y deje constancia durable sin guardar en Git todos los outputs de SQX y MT5. Es un expediente lógico, no un almacén de artefactos.

Estructura orientativa:

```
records/
├── strategies/
│   └── <strategy-id>.yaml
└── runs/
    └── <run-id>/
        ├── manifest.yaml
        ├── summary.md
        └── artifacts.sha256
```

- `strategies/` registra identidad estable, proyecto, familia, dirección, Magic Number cuando aplique, estado y relaciones entre versiones.
- `manifest.yaml` registra quién ejecutó la tarea, fechas, versiones de SQX/MT5, dataset, símbolo, timeframe, costes, configuración, etapas y URI de los artefactos.
- `summary.md` conserva conteos, métricas relevantes, incidencias, interpretación, decisión y próximo paso.
- `artifacts.sha256` relaciona cada archivo operativo con su nombre, ubicación lógica y hash, de modo que pueda comprobarse aunque se encuentre fuera de Git.

Estos registros deben ser pequeños, revisables y libres de secretos. No duplican cada celda de un reporte ni cada parámetro serializado: enlazan la configuración canónica y resumen lo necesario para reconstruir y auditar el trabajo. Cuando exista el catálogo, estos archivos podrán importarse; no deben diseñarse como una segunda base de datos permanente.

`projects/<project-slug>/platforms/`

Agrupa material versionable específico de herramientas externas. La palabra "platform" indica que MT5 y SQX son adaptadores, no el modelo central del laboratorio.

`platforms/mt5/`

Puede contener:

- código fuente MQL5;
- plantillas pequeñas;
- presets versionables;
- adaptadores de exportación;
- instrucciones específicas de MT5;
- fixtures mínimas para pruebas.

No debe contener outputs de ejecución o datos operativos. En particular, quedan fuera de `platforms/mt5/` y de Git:

- ejecutables `.ex5` y `.ex4`;
- instalaciones o copias del directorio de datos de MT5;
- históricos completos de ticks, barras o símbolos;
- cachés, agentes y logs del Strategy Tester;
- reportes completos HTML, XML, CSV, XLSX, PDF o PNG de backtests e incubaciones;
- paquetes de deployment, evidence bundles y grandes colecciones de presets generados;
- bases de datos, copias de cuentas y cualquier archivo con credenciales o datos sensibles.

Esos elementos viven en `_storage/`, en Google Drive durante el intercambio con la VPS MT5 o, más adelante, en almacenamiento de objetos y releases controlados. Git conserva el manifiesto, hash, resumen y decisión que permiten localizarlos y verificarlos.

Organización interna:

- `src/`: código MQL5 mantenido o promovido explícitamente por el equipo; el `.mq5` generado para un candidato todavía experimental permanece en storage.
- `include/`: librerías y componentes compartidos versionados.
- `presets/`: presets pequeños, seguros y sin secretos.
- `tests/`: fixtures y comprobaciones técnicas del bot.

`platforms/sqx/`

Puede contener:

- configuraciones de generación;
- plantillas pequeñas;
- scripts o snippets integrables;

- definiciones de workflows;
- adaptadores de exportación;
- fixtures mínimas.

No debe contener outputs de una minería, retest o análisis. En particular, quedan fuera de `platforms/sqx/` y de Git:

- instalaciones de StrategyQuant X y su estado interno;
- históricos completos de mercado y copias de Data Manager;
- databanks, poblaciones, generaciones y colecciones de candidatos;
- estrategias `.sqx` generadas por una corrida, salvo una fixture mínima o plantilla promovida y justificada;
- reportes completos HTML, CSV, JSON, XML, XLSX, PDF, gráficos PNG y exportaciones masivas;
- bases SQLite, cachés, logs, temporales y snapshots completos de la instalación;
- `.mq5` exportados de candidatos y cualquier `.ex5` compilado.

Sí pueden versionarse configuraciones pequeñas y canónicas como `.cfx`, `.sqb`, XML o plantillas `.sqx` cuando sean necesarias para reproducir un workflow, estén documentadas, no contengan secretos y su hash quede registrado. Que un formato sea admitido no significa que toda exportación de ese formato deba entrar en Git.

Organización interna:

- `export-profiles/`: perfiles controlados que determinan cómo se exporta un candidato.
- `templates/`: plantillas pequeñas y versionables.
- `adapters/`: reglas o scripts específicos para transformar la exportación.

7. `apps/` — Programas ejecutables

Cada elemento de `apps/` es una unidad que una persona o proceso puede ejecutar o desplegar. Puede depender de `packages/`, pero no debe duplicar su lógica central.

`apps/api/`

Expone operaciones controladas del laboratorio a otros clientes.

Responsabilidades:

- validar solicitudes;
- autenticar y autorizar;
- ejecutar casos de uso;
- devolver respuestas tipadas;
- registrar auditoría y errores.

No debe contener lógica específica de una interfaz ni permitir SQL arbitrario. En el futuro, MCP se conectará a esta capa o a los mismos servicios de aplicación.

`apps/cli/`

Ofrece comandos para personas, scripts y automatizaciones.

Ejemplos:

```
tradinglab project create
tradinglab experiment register
tradinglab run import-mt5
tradinglab artifact verify
```

La CLI debe producir códigos de salida correctos, mensajes claros y, cuando sea necesario, salida estructurada. Debe usar los mismos casos de uso que la API.

`apps/worker/`

Ejecuta tareas en segundo plano o de larga duración:

- importación de reportes;
- cálculo de hashes;
- normalización de métricas;
- controles de integridad;
- generación de informes;
- procesamiento de colas.

Durante el piloto puede no existir. Se introducirá cuando los procesos ya no deban ejecutarse sincrónicamente.

`apps/mt5-runner/`

Contiene la aplicación instalable que opera localmente en la VPS MT5 externa. Sus funciones incluyen validar paquetes, solicitar aprobación local, compilar, ejecutar backtests, administrar incubaciones y producir evidence bundles.

No contiene secretos, estrategias privadas ajenas al job ni acceso al catálogo de TradingLab. Se distribuye mediante paquetes versionados, no mediante una rama permanente por servidor. El workflow se describe en `tradinglab-mt5-runner-workflow.md`.

8. `packages/` — Capacidades reutilizables

Los paquetes contienen lógica compartida por varias aplicaciones. No son ejecutables autónomos y deben evitar dependencias innecesarias de infraestructura.

`packages/domain/`

Implementa el lenguaje y reglas centrales de TradingLab:

- entidades y value objects;
- estados y transiciones;
- validación de manifiestos;
- reglas de identidad e inmutabilidad;
- contratos de repositorios o servicios.

No debería conocer detalles de HTTP, interfaz gráfica, MCP o rutas de una máquina.

`packages/ingestion/`

Transforma outputs externos en información interna validada.

Incluye:

- parsers de MT5 y SQX;
- detección de formatos y versiones;
- modelo intermedio;
- normalización de unidades;
- validaciones y deduplicación;
- fixtures y pruebas de regresión.

Debe conservar el archivo original y declarar la versión del parser utilizada.

`packages/bot-pipeline/`

Implementa las capacidades reutilizables que llevan una estrategia SQX hasta un bot MT5 incubable:

- preparación y normalización;
- resolución de componentes MQL5;
- invocación controlada de MetaEditor;
- generación de build manifests;
- validación de gates;
- empaquetado de `.ex5`, preset y evidencias;
- verificación previa al deployment.

No contiene ejecutables compilados ni credenciales. Esos outputs viven en storage y sus metadatos en el catálogo.

`packages/exchange-contracts/`

Define los contratos compartidos entre las dos VPS:

- `RunnerRelease`;
- `JobPackage`;
- `Acknowledgement`;
- `EvidenceBundle`;
- versiones de esquema;
- validadores y códigos de error.

Debe poder utilizarse en ambos lados sin otorgar acceso a la base ni a otros componentes internos.

`packages/reporting/`

Genera vistas y entregables a partir de datos autoritativos:

- tablas comparativas;
- resúmenes de experimentos;
- reportes PDF/HTML;
- datasets de análisis derivados.

Los reportes son derivados: no deben convertirse silenciosamente en la única fuente de un resultado.

9. `db/` — Evolución del catálogo

`db/` contiene la definición reproducible de la base, no los archivos físicos de PostgreSQL ni copias de producción.

`db/migrations/`

Cada migración representa una transformación ordenada del esquema. Debe tener identificador, descripción y, cuando sea seguro, una operación inversa.

Las migraciones se revisan como código. No se modifica una migración que ya fue aplicada en entornos compartidos; se crea otra.

`db/seeds/`

Contiene datos iniciales no sensibles y repetibles:

- definiciones estándar de métricas;
- estados o catálogos controlados;
- datos mínimos de desarrollo.

No contiene resultados reales, credenciales, información contractual ni copias de producción.

`db/README.md`

Explica:

- motor y versiones compatibles;
- cómo crear y actualizar la base;
- cómo ejecutar tests de migración;
- estrategia de respaldo y restauración;
- convenciones de nombres;
- procedimiento de recuperación ante fallos.

10. `infra/` — Entornos y despliegue

Contiene infraestructura declarativa y reproducible:

- Dockerfiles y Compose;
- configuración de servicios;
- plantillas de CI/CD;
- observabilidad;
- infraestructura como código;
- scripts de despliegue controlados.

No contiene volúmenes reales, estados de producción ni secretos. Debe ser posible revisar qué infraestructura cambia mediante un diff de Git.

11. `tools/` — Utilidades de mantenimiento

Contiene herramientas para desarrollar o mantener el repositorio, no capacidades de negocio principales.

Ejemplos:

- verificador de hashes;
- validador de convenciones;
- generador de fixtures;
- comprobación de enlaces;
- utilidades de desarrollo local.

Regla de promoción:

- si una utilidad se convierte en interfaz estable para usuarios, pasa a `apps/cli/`;
- si implementa lógica reutilizable, pasa a `packages/`;
- si es un procedimiento de agente, puede ser script de una skill;
- si solo sirve una vez, debe eliminarse o documentarse claramente.

12. `_storage/` — Área local no versionada y espacio de trabajo humano

Aunque no forma parte del contenido de Git, durante el piloto puede existir un directorio `_storage/` ignorado:

```
_storage/
├── datasets/
├── artifacts/
├── projects/
│   └── <project-slug>/
│       ├── inbox/
│       ├── strategies/
│       └── runs/
│           └── <run-id>/
│               ├── inputs/
│               ├── sqx/
│               ├── mt5/
│               └── derived/
├── inbox/
└── quarantine/
```

- `datasets/`: snapshots de datos de mercado.
- `artifacts/`: reportes y outputs registrados.
- `projects/`: espacio de trabajo reconocible para la operación manual, separado por proyecto, estrategia y ejecución.
- `inbox/`: archivos pendientes de validar e importar.
- `quarantine/`: archivos rechazados, corruptos o de formato desconocido.

Dentro de una ejecución, `inputs/` preserva lo recibido; `sqx/` contiene exports y reportes de SQX; `mt5/` contiene builds y reportes de MT5; y `derived/` contiene comparativas, hojas de trabajo o bases reconstruidas. Los originales no se sobrescriben. Un archivo derivado debe declarar sus fuentes.

Esta estructura permite conservar el modo de trabajo manual actual: el socio puede abrir una carpeta de estrategia o ejecución y encontrar todos los insumos y resultados relacionados. La diferencia es que la identidad, los parámetros importantes, los hashes y la decisión quedan también registrados en `projects/<slug>/records/`, en vez de depender del nombre informal de la carpeta.

El contenido de `_storage/` requiere backup separado: `.gitignore` evita subirlo a Git, pero **no constituye una copia de seguridad**. La base guarda URI y hash; mover el contenido a almacenamiento de objetos no debe exigir cambiar el modelo de dominio.

12.1 Convivencia con el procedimiento manual de TradingLab 1.0

El análisis del workspace `C:\TradingLab` mostró que el procedimiento real agrupa por proyecto, tanda y estrategia:

- configuraciones Builder/Retester y snapshots de tareas SQX;
- candidatos `.sqx`, fuentes `.mq5`, presets `.set` y compilados `.ex5`;
- backtests SQX y MT5 en HTML, XML, CSV, XLSX y PNG;
- hojas de incubación identificadas por estrategia o Magic Number;
- scripts de consolidación, bases SQLite y resúmenes de selección;
- documentación, decisiones, errores detectados y reportes de sesión.

Este expediente autocontenido ayuda al operador y no debe eliminarse abruptamente. TradingLab 2.0 separa su **plano de control** de su **plano de artefactos**:

Elemento observado en 1.0	Destino en 2.0	Regla
Objetivo, estado, responsables y criterios	`projects/<slug>/README.md` y `docs/`	Versionado
Parámetros canónicos de estrategia, build o incubación	`projects/<slug>/configs/`	Versionado y enlazado por hash
Identidad, Magic Number, versión, run, métricas y decisión	`projects/<slug>/records/`	Versionado durante el piloto
Plantilla SQX o configuración reutilizable promovida	`platforms/sqx/`	Solo pieza pequeña, explicada y estable
Fuente MQL5 mantenida o promovida	`platforms/mt5/src/`	Versionada; el generado experimental no entra
Candidatos `.sqx` / `.mq5`, `.ex5`, reportes y gráficos	`_storage/projects/<slug>/...`	No versionado; URI y SHA-256 registrados
Históricos, databanks y resultados masivos	`_storage/datasets/` o storage externo	Nunca en `platforms/`
Scripts estables de ingestión o comparación	`packages/ingestion/`, `packages/reporting/` o `tools/`	Se promueven según su responsabilidad
Workbook o SQLite reconstruible de incubación	`_storage/.../derived/`	Derivado; registrar fuentes y versión del proceso

Elemento observado en 1.0	Destino en 2.0	Regla
Lección reutilizable	`docs/knowledge/`	Promoción explícita, conservando origen
Cronología de lo realizado	`docs/sessions/`	No sustituye el registro del run

Los identificadores humanos actuales pueden mantenerse como aliases, por ejemplo `11112` o `3.14.77`, pero no deben ser la única identidad. Cada estrategia y cada ejecución recibirán identificadores estables; generación, ranking, tanda y Magic Number serán metadata consultable.

La migración será incremental:

1. congelar TradingLab 1.0 como fuente histórica de solo lectura y crear un inventario con hashes;
2. elegir un proyecto piloto activo, sin mover inicialmente los demás;
3. registrar estrategias y runs activos en `records/` y copiar sus artefactos a `_storage/` preservando originales;
4. validar que el socio pueda completar el mismo checklist manual y localizar cada evidencia;
5. automatizar primero la generación de manifiestos, hashes e índices, no la decisión de trading;
6. importar los registros al catálogo cuando este exista y mantener redirects o mapas hacia los IDs antiguos.

No se hará una migración masiva basada solo en extensiones. Un `.mq5`, `.sqx`, `.xml` o `.csv` puede ser fuente canónica, fixture, resultado o temporal según su procedencia y función; primero se clasifica y después se mueve o promueve.

13. Cómo decidir dónde colocar algo

1. ¿Es una decisión importante? → `docs/adr/`.
2. ¿Describe la arquitectura vigente? → `docs/architecture/`.
3. ¿Es conocimiento aplicable a varios proyectos? → `docs/knowledge/`.
4. ¿Es historia de una sesión? → `docs/sessions/`.
5. ¿Solo aplica a un proyecto? → `projects/<slug>/...`.
6. ¿Es una aplicación ejecutable? → `apps/`.
7. ¿Es lógica compartida? → `packages/`.
8. ¿Modifica o inicializa el esquema? → `db/`.
9. ¿Define despliegue o entorno? → `infra/`.
10. ¿Es una utilidad interna? → `tools/`.
11. ¿Es un manifiesto, resumen o inventario pequeño de una ejecución manual? → `projects/<slug>/records/` durante el piloto.
12. ¿Es pesado, generado, binario, un histórico o un reporte completo? → `_storage/`, con URI y hash en el registro o catálogo.

14. Errores que la estructura intenta evitar

- Usar carpetas llamadas `misc`, `temp` o `otros` como destino permanente.
- Duplicar la misma lógica en API, CLI y workers.

- Mezclar código fuente con outputs de backtests.
- Guardar secretos dentro de configuraciones versionadas.
- Convertir la bitácora en base de datos manual.
- Crear un `AGENTS.md` enorme con procedimientos que deberían ser skills.
- Crear directorios vacíos "por si acaso".
- Atar rutas o configuraciones a un único ordenador.
- Confundir reportes derivados con datos autoritativos.

15. Estructura mínima para comenzar

Después de aprobar la arquitectura no es necesario crear todo el árbol. La primera versión puede limitarse a:

```
README.md
AGENTS.md
.gitignore
.env.example
docs/
├── architecture/
├── adr/
└── sessions/
projects/
└── <pilot-project>/
    ├── README.md
    ├── configs/
    │   ├── records/
    └── platforms/
db/
└── migrations/
```

`apps/`, la mayoría de `packages/`, `infra/`, `tools/` y las skills aparecerán al implementar el flujo piloto. `packages/bot-pipeline/` se creará cuando comience la automatización real de preparación y compilación. Esta disciplina evita diseñar una estructura vacía antes de conocer las necesidades reales.

16. Resumen

La estructura separa conocimiento, investigación, software ejecutable, lógica reutilizable, esquema de datos e infraestructura. Esa separación permite que TradingLab crezca sin depender de convenciones tácitas y facilita que personas, automatizaciones y agentes trabajen con las mismas reglas.